

Session 09 — Assignment

Topics: Primary Constructors, Records, Singleton, `var` & `dynamic`, Anonymous Types, Extension Methods, Unit Testing (xUnit)

Answer all questions in a new solution named `Session09-Assignment`. For the coding questions, create a console project and an xUnit test project (the same structure we used in the session).

Part 1 — Primary Constructor & Records

A hospital system needs a patient model.

Q1. Create a class `Patient` with a **primary constructor** that takes: `Id (int)`, `FullName (string)`, `PhoneNumber (string)`, `MedicalHistory (string)`.

- Override `ToString()` to return: `Id: {Id} :: FullName: {FullName} :: PhoneNumber: {PhoneNumber} :: MedicalHistory: {MedicalHistory}`

Q2. In `Main`, create two `Patient` objects with the **same** values, then:

1. Print `GetHashCode()` of both. Are they equal? Why?
2. Print `patient01.Equals(patient02)`. What is the result and why?
3. Assign `patient01 = patient02`, then check `Equals` again. What changed?

Q3. Medical history is sensitive. Create a positional **record** `PatientDto` that exposes only: `Id`, `FullName`, `PhoneNumber`.

- Repeat the comparisons from Q2 with two identical `PatientDto` objects.
- In a comment, explain the difference you observed between the class and the record.

Q4. Create a static class `PatientMapper` with a static method `MapFromModelToDto(Patient patient)` that returns a `PatientDto`. Use it in `Main` to convert a `Patient` and print the result.

Part 2 — Singleton

Q5. Apply the **Singleton** pattern to a class `AppLogger`:

- Make the constructor **private**.
- Add a private static field to hold the single instance (starts as `null`).
- Add a public static method `GetLogger()` that creates the instance only on the first call and returns the **same** instance every time.

Q6. In `Main`, call `AppLogger.GetLogger()` four times and print the `GetHashCode()` of each result. What do you observe? Explain **why** in a comment.

Part 3 — var & dynamic

Q7. Do the following in `Main`:

1. Declare a `Patient` using target-typed `new` (`Patient p = new(...)`).
 2. Declare another `Patient` using `var`.
 3. Declare a dynamic variable, assign a `Patient` to it, and print it.
 4. In a comment, explain the difference between `var` and `dynamic`: when is each type resolved — compile time or run time?
-

Part 4 — Anonymous Types

Q8. Create two anonymous objects `doctor01` and `doctor02` with identical properties and values:

```
Name = "Sara", Specialty = "Cardiology", ExperienceYears = 8, Salary = 25_000
```

Then print:

1. `doctor01.Name` and `doctor01.Specialty`
2. `GetHashCode()` of both objects
3. `doctor01.GetType()`
4. `doctor01.Equals(doctor02)`
5. `doctor01.ToString()`

In a comment, compare anonymous-type equality with **class** equality and **record** equality.

Part 5 — Extension Methods

Q9. Create a static class `TextHelper` with these **extension methods** on `string`:

1. `IsShorterThan(this string value, int length)` — returns `true` if `value.Length` is less than `length`; otherwise `false`.
2. `Repeat(this string value, int times)` — returns the string repeated `times` times. Example: `"Ha".Repeat(3)` → `"HaHaHa"`.

Bonus: write `IsShorterThan` as a normal static method first, then convert it into an extension method using the `this` keyword — exactly like we did in the session with `IsLongerThan`.

Q10. In `Main`, call `"Stethoscope".IsShorterThan(5)` and `"Ab".Repeat(4)` and print the results.

Part 6 — Unit Testing (xUnit)

A library system needs a fine calculator.

Q11. Create a class `FineCalculator` with these methods:

- `int Add(int a, int b)`
- `int Subtract(int a, int b)`
- `int Multiply(int a, int b)`
- `int Divide(int a, int b)` — **must throw** `DivideByZeroException` **when** `b == 0`

Q12. In a separate xUnit test project, create a test class `FineCalculatorTests`. Create the `FineCalculator` instance in the **constructor** (stored in a `readonly` field), follow the **AAA** pattern (Arrange – Act – Assert), and write:

#	Test name	Type	Assertion
1	<code>Add_ShouldReturnCorrectSum</code>	<code>[Fact]</code>	<code>Assert.Equal</code>
2	<code>Add_ShouldReturnNotEqual</code>	<code>[Fact]</code>	<code>Assert.NotEqual</code>
3	<code>Subtract_ShouldReturnCorrectDifference</code>	<code>[Fact]</code>	<code>Assert.Equal</code>
4	<code>Multiply_ShouldReturnCorrectResult</code>	<code>[Theory] + 3</code> <code>×</code>	<code>Assert.Equal</code>
		<code>[InlineData]</code> (include a negative-number-case)	
		<code>[Theory] + 3</code> <code>×</code>	
5	<code>Multiply_ShouldReturnNotEqual</code>	<code>[Theory] + 3</code> <code>×</code>	<code>Assert.NotEqual</code>
6	<code>Divide_ByZero_ShouldThrowException</code>	<code>[Theory] + 3</code> <code>×</code>	<code>Assert.Throws<DivideByZeroException></code>
		<code>[InlineData]</code>	

Submission Checklist

- ☐ Solution contains a console project and a test project
- ☐ All `Main` questions answered with explanatory comments
- ☐ All 6 tests pass in Test Explorer
- ☐ Code pushed to your GitHub repository

